

Laboratorio 8

Marcos Díaz (221102), Daniel Machic (22118), Maria Jose Ramírez (221051)

GitHub: <https://github.com/MarcosDiaz1409/Lab08-IA.git>

Ejercicio 1

a) ($P(X_1 = 1, X_2 = 0)$)

Sumar todas las filas donde:

- ($X_1=1$)
- ($X_2=0$)

$$P(X_1 = 1, X_2 = 0) = 0.10 + 0.05$$

$$P(X_1 = 1, X_2 = 0) = \mathbf{0.15}$$

b) ($P(X_3 = 0)$)

Sumar todas las filas donde ($X_3=0$):

$$P(X_3 = 0) = 0.05 + 0.10 + 0.40 + 0.10$$

$$P(X_3 = 0) = 0.65$$

$$P(X_3 = 0) = \mathbf{0.65}$$

c) ($P(X_2 = 1 \mid X_3 = 1)$)

Usar la fórmula de probabilidad condicional:

$$P(X_2 = 1|X_3 = 1) = \frac{P(X_2 = 1, X_3 = 1)}{P(X_3 = 1)}$$

Calcular el numerador:

$$P(X_2 = 1, X_3 = 1) = 0.20 + 0.00$$

$$P(X_2 = 1, X_3 = 1) = 0.20$$

Calcular el denominador:

$$P(X_3 = 1) = 0.10 + 0.05 + 0.20 + 0.00$$

$$P(X_3 = 1) = 0.35$$

Entonces:

$$P(X_2 = 1|X_3 = 1) = \frac{0.20}{0.35}$$

$$P(X_2 = 1|X_3 = 1) \approx \mathbf{0.5714}$$

d) (P(X₁ = 0 | X₂ = 1, X₃ = 1))

Usar:

$$P(X_1 = 0|X_2 = 1, X_3 = 1) = \frac{P(X_1 = 0, X_2 = 1, X_3 = 1)}{P(X_2 = 1, X_3 = 1)}$$

El numerador es:

$$P(X_1 = 0, X_2 = 1, X_3 = 1) = 0.20$$

El denominador es:

$$P(X_2 = 1, X_3 = 1) = 0.20 + 0.00$$

$$P(X_2 = 1, X_3 = 1) = 0.20$$

Entonces:

$$P(X_1 = 0|X_2 = 1, X_3 = 1) = \frac{0.20}{0.20}$$

$$P(X_1 = 0|X_2 = 1, X_3 = 1) = \mathbf{1}$$

e) ($P(X_1 = 0, X_2 = 1 \mid X_3 = 1)$)

Usar:

$$P(X_1 = 0, X_2 = 1 \mid X_3 = 1) = \frac{P(X_1 = 0, X_2 = 1, X_3 = 1)}{P(X_3 = 1)}$$

Calcular el numerador:

$$P(X_1 = 0, X_2 = 1, X_3 = 1) = 0.20$$

Calcular el denominador:

$$P(X_3 = 1) = 0.35$$

Entonces:

$$P(X_1 = 0, X_2 = 1 \mid X_3 = 1) = \frac{0.20}{0.35}$$

$$P(X_1 = 0, X_2 = 1 \mid X_3 = 1) \approx \mathbf{0.5714}$$

Ejercicio 2

a) $P(T = 1 \mid R = 0)$

$$P(T=1 \mid R=0) = \mathbf{0.1}$$

b) $P(T = 1)$

$$P(T=1) = P(T=1 \mid R=1)P(R=1) + P(T=1 \mid R=0)P(R=0)$$

$$= (0.8)(0.1) + (0.1)(0.9)$$

$$= 0.08 + 0.09$$

$$P(T=1) = \mathbf{0.17}$$

c) $P(L = 1 \mid R = 0, T = 1)$

$$P(L=1|R=0,T=1) = P(L=1|T=1)$$

De la tabla:

$$P(L=1|R=0,T=1) = \mathbf{0.3}$$

d) $P(L = 1 \mid T = 1)$

También se obtiene directamente de la tabla $P(L|T)$:

$$P(L=1|T=1) = \mathbf{0.3}$$

e) $P(L = 1 \mid R = 0)$

$$P(L=1|R=0) = \sum_t P(L=1|T=t) P(T=t|R=0)$$

Expandimos:

$$= P(L=1|T=1)P(T=1|R=0) + P(L=1|T=0)P(T=0|R=0)$$

Sustituyendo:

$$= (0.3)(0.1) + (0.1)(0.9)$$

$$= 0.03 + 0.09$$

$$P(L=1|R=0) = \mathbf{0.12}$$

f) $P(T = 1 \mid L = 0)$

Calculamos $P(L=0)$:

$$P(L=0) = P(L=0|T=1)P(T=1) + P(L=0|T=0)P(T=0)$$

Como $P(T=0) = 1 - 0.17 = 0.83$, entonces:

$$P(L=0) = (0.7)(0.17) + (0.9)(0.83)$$

$$= 0.119 + 0.747$$

$$= 0.866$$

Finalmente:

$$P(T=1|L=0) = (0.7)(0.17) / 0.866$$

$$= 0.119 / 0.866 \approx 0.1374$$

$$P(T=1|L=0) \approx \mathbf{0.137}$$

Ejercicio 3

In [1]: `import pandas as pd`

```
# P(G)
P_G = {
    '+': 0.1,
    '-': 0.9
}

# P(M | G)
P_M_given_G = {
    ('+', '+'): 0.667,
    ('-', '+'): 0.333,
    ('+', '-'): 0.25,
    ('-', '-'): 0.75
}

# P(B | M)
P_B_given_M = {
    ('+', '+'): 0.4,
    ('-', '+'): 0.6,
    ('+', '-'): 0.2,
    ('-', '-'): 0.8
}

# P(C | M)
P_C_given_M = {
    ('+', '+'): 0.25,
    ('-', '+'): 0.75,
    ('+', '-'): 0.5,
    ('-', '-'): 0.5
}

results = []

for G in ['+', '-']:
    for M in ['+', '-']:
        for B in ['+', '-']:
            for C in ['+', '-']:

                prob = (
                    P_G[G]
                    * P_M_given_G[(M, G)]
                    * P_B_given_M[(B, M)]
                    * P_C_given_M[(C, M)]
                )

                results.append([G, M, B, C, prob])
```

```

df = pd.DataFrame(
    results,
    columns=['G', 'M', 'B', 'C', 'P(G,M,B,C)']
)

# Mostrar tabla
print(df)

from fractions import Fraction

print("\nTabla con fracciones aproximadas:\n")

for index, row in df.iterrows():

    frac = Fraction(row['P(G,M,B,C)']).limit_denominator()

    print(
        f"G={row['G']} "
        f"M={row['M']} "
        f"B={row['B']} "
        f"C={row['C']} "
        f"-> {frac}"
    )

```

	G	M	B	C	P(G,M,B,C)
0	+	+	+	+	0.006670
1	+	+	+	-	0.020010
2	+	+	-	+	0.010005
3	+	+	-	-	0.030015
4	+	-	+	+	0.003330
5	+	-	+	-	0.003330
6	+	-	-	+	0.013320
7	+	-	-	-	0.013320
8	-	+	+	+	0.022500
9	-	+	+	-	0.067500
10	-	+	-	+	0.033750
11	-	+	-	-	0.101250
12	-	-	+	+	0.067500
13	-	-	+	-	0.067500
14	-	-	-	+	0.270000
15	-	-	-	-	0.270000

Tabla con fracciones aproximadas:

G=+ M=+ B=+ C=+ -> 667/100000
 G=+ M=+ B=+ C=- -> 2001/100000
 G=+ M=+ B=- C=+ -> 2001/200000
 G=+ M=+ B=- C=- -> 6003/200000
 G=+ M=- B=+ C=+ -> 333/100000
 G=+ M=- B=+ C=- -> 333/100000
 G=+ M=- B=- C=+ -> 333/25000
 G=+ M=- B=- C=- -> 333/25000
 G=- M=+ B=+ C=+ -> 9/400
 G=- M=+ B=+ C=- -> 27/400
 G=- M=+ B=- C=+ -> 27/800
 G=- M=+ B=- C=- -> 81/800
 G=- M=- B=+ C=+ -> 27/400
 G=- M=- B=+ C=- -> 27/400
 G=- M=- B=- C=+ -> 27/100
 G=- M=- B=- C=- -> 27/100

La probabilidad conjunta se calcula utilizando:

$$P(G, M, B, C) = P(G) P(M|G) P(B|M) P(C|M)$$

Cálculo

Para:

$$G = +, M = +, B = +, C = +$$

se tiene:

$$P(+, +, +, +) = P(G = +) P(M = +|G = +) P(B = +|M = +) P(C = +|M = +)$$

Sustituyendo valores:

$$= (0.1)(0.667)(0.4)(0.25)$$

= 0.00667

Tabla de probabilidades conjuntas

				$P(G,$
G	M	B	C	$M, B,$
				$C)$
+	+	+	+	0.006670
+	+	+	-	0.020010
+	+	-	+	0.010005
+	+	-	-	0.030015
+	-	+	+	0.003330
+	-	+	-	0.003330
+	-	-	+	0.013320
+	-	-	-	0.013320
-	+	+	+	0.022500
-	+	+	-	0.067500
-	+	-	+	0.033750
-	+	-	-	0.101250
-	-	+	+	0.067500
-	-	+	-	0.067500
-	-	-	+	0.270000
-	-	-	-	0.270000

Tabla con fracciones aproximadas

G	M	B	C	Fracción
+	+	+	+	$\frac{667}{100000}$
+	+	+	-	$\frac{2001}{100000}$
+	+	-	+	$\frac{2001}{200000}$
+	+	-	-	$\frac{6003}{200000}$
+	-	+	+	$\frac{333}{100000}$

G	M	B	C	Fracción
+	-	+	-	$\frac{333}{100000}$
+	-	-	+	$\frac{333}{25000}$
+	-	-	-	$\frac{333}{25000}$
-	+	+	+	$\frac{9}{400}$
-	+	+	-	$\frac{27}{400}$
-	+	-	+	$\frac{27}{800}$
-	+	-	-	$\frac{81}{800}$
-	-	+	+	$\frac{27}{400}$
-	-	+	-	$\frac{27}{400}$
-	-	-	+	$\frac{27}{100}$
-	-	-	-	$\frac{27}{100}$

Verificación

La suma de todas las probabilidades conjuntas es:

$$\sum_{G,M,B,C} P(G, M, B, C) = 1$$

lo cual confirma que la distribución conjunta es válida.

Ejercicio 4

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

GRID_SIZE = 10

# Prior uniforme
belief = np.ones((GRID_SIZE, GRID_SIZE))
belief = belief / belief.sum()

# Leer distribución de sensores
sensor_df = pd.read_csv("Sensor_Color_Distribution.csv")

print(sensor_df.head())
```

```

# Función para obtener probabilidad
def sensor_probability(distance, observed_color):

    # Filtrar por distancia
    row = sensor_df[sensor_df["dist"] == distance]

    if len(row) == 0:
        return 1e-6

    return float(row[observed_color].values[0])

def manhattan_distance(x1, y1, x2, y2):
    return abs(x1 - x2) + abs(y1 - y2)

# Actualización Bayesiana
def update_belief(belief, evidence_position, observed_color):

    new_belief = np.zeros_like(belief)

    sensor_x, sensor_y = evidence_position

    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):

            # Distancia entre fantasma hipotético y sensor
            d = manhattan_distance(i, j, sensor_x, sensor_y)

            # Likelihood
            likelihood = sensor_probability(d, observed_color)

            # Bayes
            new_belief[i, j] = likelihood * belief[i, j]

    # Normalizar
    new_belief = new_belief / new_belief.sum()

    return new_belief

```

	dist	R	O	Y	G	B
0	0	0.30	0.00	0.00	0.0	0.0
1	1	0.27	0.01	0.00	0.0	0.0
2	2	0.21	0.03	0.00	0.0	0.0
3	3	0.13	0.08	0.00	0.0	0.0
4	4	0.06	0.15	0.01	0.0	0.0

Inciso a)

In []: *# Mostrar plots colorizados*

```

evidences = [
    ((2, 3), "R"),
    ((7, 8), "G"),
    ((5, 5), "Y")
]

```

```
current_belief = belief.copy()

step = 1

for position, color in evidences:

    current_belief = update_belief(
        current_belief,
        position,
        color
    )

    plt.figure(figsize=(6,6))

    plt.imshow(current_belief, cmap="hot")

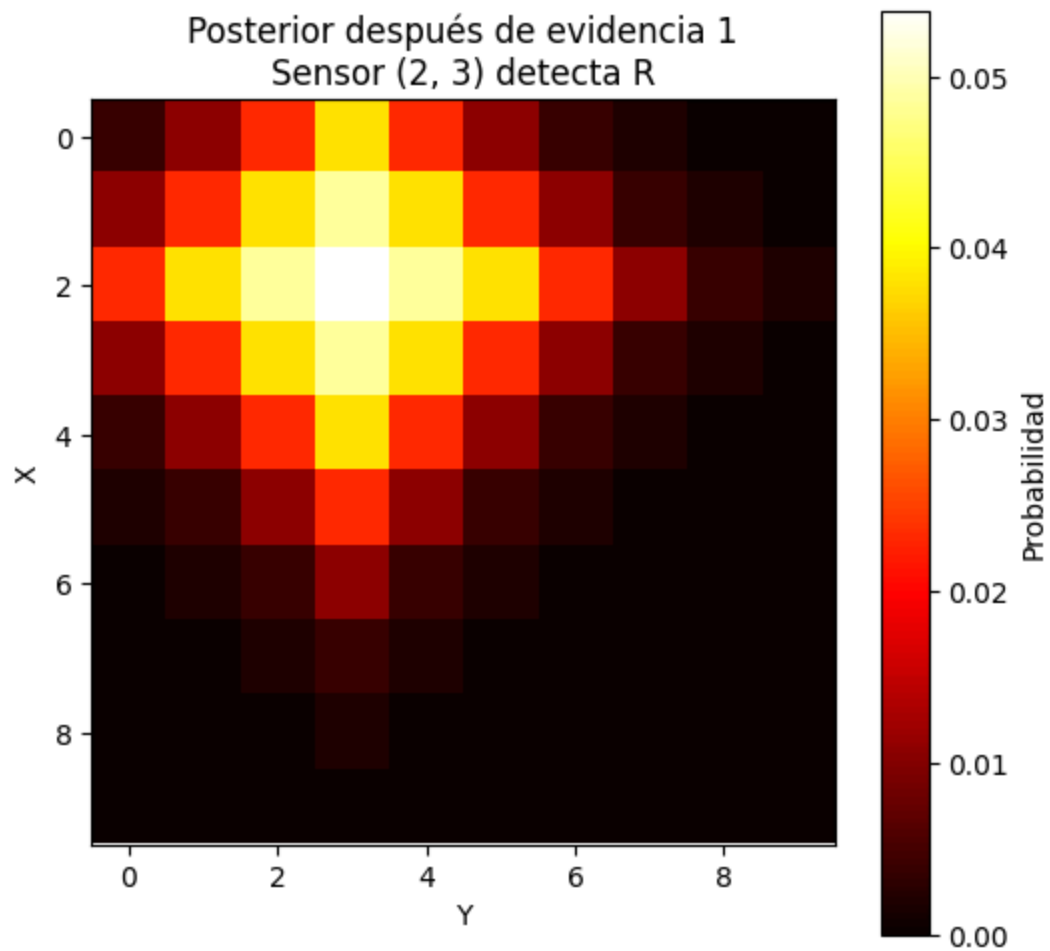
    plt.colorbar(label="Probabilidad")

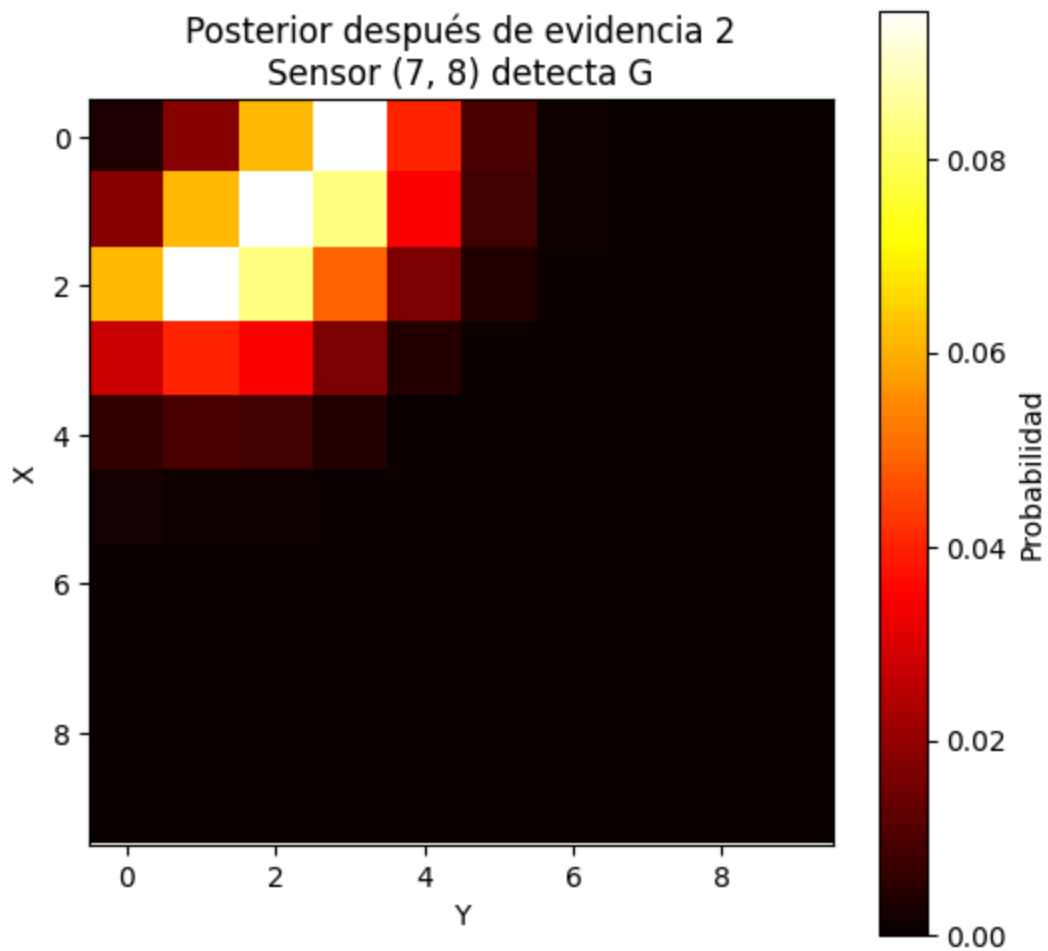
    plt.title(
        f"Posterior después de evidencia {step}\n"
        f"Sensor {position} detecta {color}"
    )

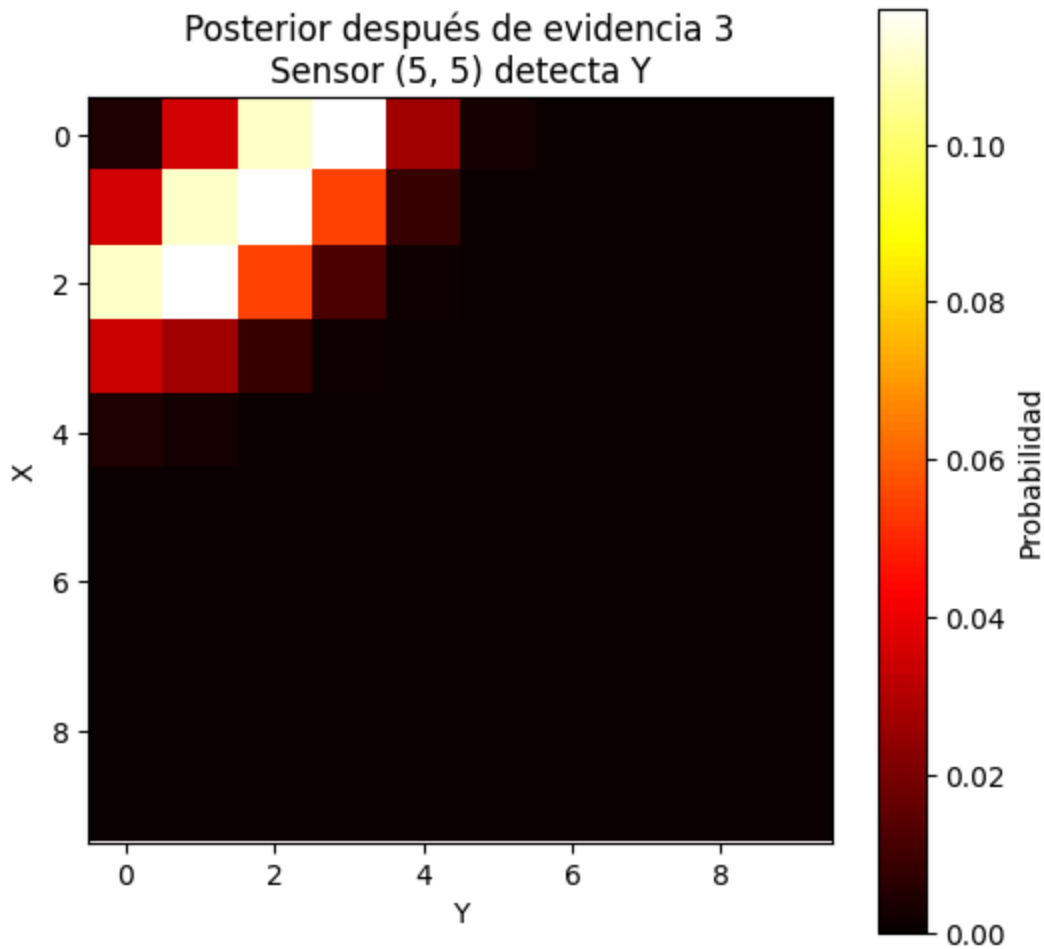
    plt.xlabel("Y")
    plt.ylabel("X")

    plt.show()

    step += 1
```







Los mapas de calor muestran cómo la distribución de probabilidad del fantasma cambia conforme se agrega nueva evidencia de los sensores. Inicialmente la probabilidad era uniforme en todas las celdas, pero después de cada observación, la probabilidad comienza a concentrarse en regiones específicas del tablero. Las zonas más claras representan posiciones con mayor probabilidad, mientras que las zonas oscuras indican posiciones poco probables o prácticamente descartadas.

A medida que se incorporan más evidencias, la incertidumbre disminuye y la distribución posterior se vuelve más precisa. En la tercera evidencia puede observarse que la probabilidad queda concentrada en muy pocas celdas, indicando que el modelo tiene una mayor certeza sobre la posible ubicación del fantasma.

Inciso b)

```
In [ ]: # Encontrar La posición más probable (MAP)

# Obtener celda con máxima probabilidad
max_position = np.unravel_index(
    np.argmax(current_belief),
    current_belief.shape
)
```

```

max_position = (int(max_position[0]), int(max_position[1]))
max_probability = float(current_belief[max_position])

print("Celda más probable del fantasma:")
print(max_position)

print("\nProbabilidad asociada:")
print(round(max_probability, 4))

# Heatmap
plt.figure(figsize=(6,6))

plt.imshow(current_belief, cmap="hot")

plt.colorbar(label="Probabilidad")

plt.scatter(
    max_position[1],
    max_position[0],
    marker='X',
    s=200
)

plt.title("Estimación MAP de la ubicación del fantasma")

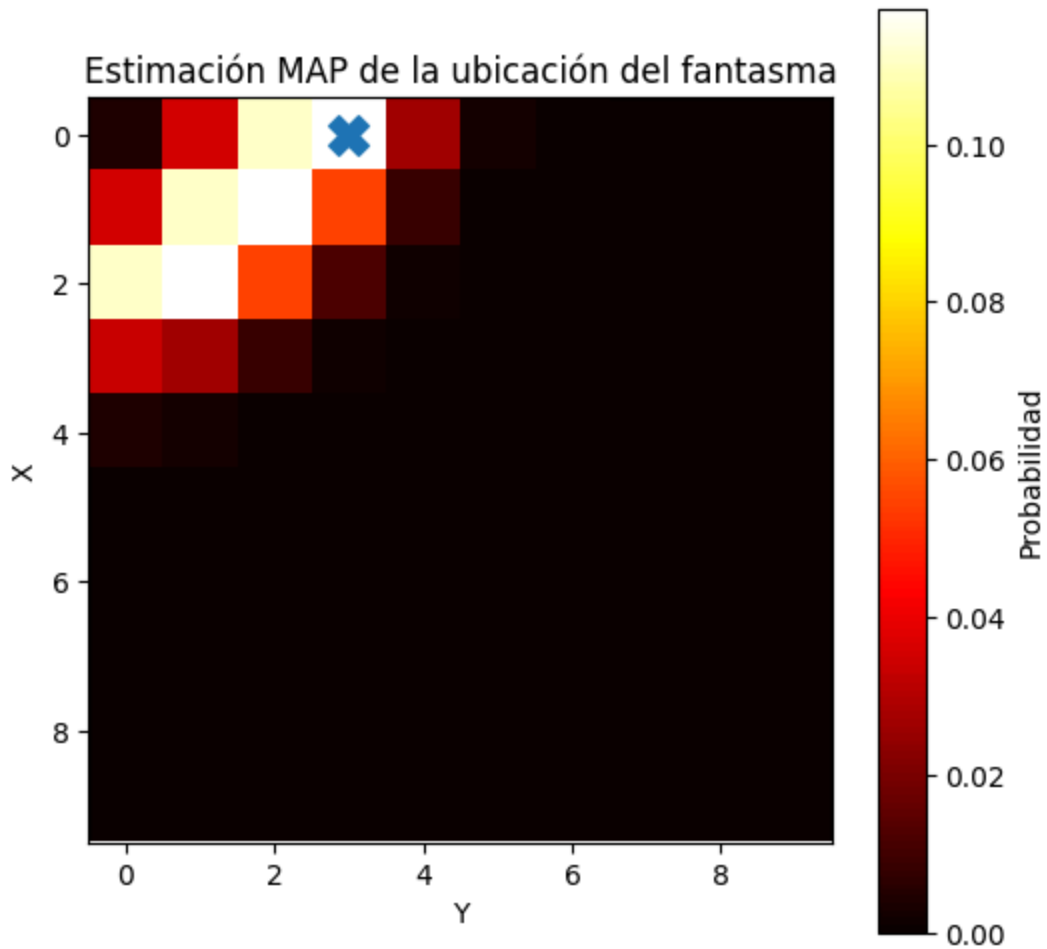
plt.xlabel("Y")
plt.ylabel("X")

plt.show()

```

Celda más probable del fantasma:
(0, 3)

Probabilidad asociada:
0.1173



La estimación MAP identifica la celda **(0,3)** como la posición más probable del fantasma, con una probabilidad aproximada de **0.1173**. En el mapa de calor, esta posición corresponde a la región más clara y marcada con una "X", indicando que es la ubicación más consistente con toda la evidencia proporcionada por los sensores después de realizar la actualización bayesiana.